Grobdesign - Kopie mit Code für Diagramme

Projekttitel	Feels Like
Erstellt am	14. November 2025
Zuletzt aktualisiert von	Julian Stengele
Zuletzt aktualisiert um	17. Januar 2026 10:38
Überprüfer/-innen	Julian Stengele Isabella Schwarz Sophie Lazarjan Atu Joey
Erstellt von	Julian Stengele
Kategorie	Strategiedokument

Versionsverwaltung

Version	Datum	Änderungen
V 1.0.0	14. November 2025	Initialversion

Analyseklassenmodell

```
classDiagram

%% ==========================================
%% 1. ENTITY-KLASSEN (Domain-Daten)
%% ==========================================

class Athlet {
    -name : String
    -email : String
```

```
    -passwortHash : String
    -status : AthletStatus     %% eingeladen / aktiv / inak
tiv
}

class Trainer {
    -name : String
    -email : String
}

class Trainingseinheit {
    -datum : Date
    -sportart : String
    -dauer : int
    -distanz : double
}

class Wettbewerb {
    -datum : Date
    -name : String
    -sportart : String
    -dauer : int
    -platzierung : int
    -status : String        %% z.B. abgeschlossen / geplant
    -zeit : double
}

class StatistikFilter {
    -sportart : String
    -zeitraum : Zeitraum
}

class RankingFilter {
    -sportart : String
    -zeitraum : Zeitraum
    -distanz : double
    -leistungskennzahl : String
}
```

```
class CSVDatei {
    -inhalt : String
}

%% ==========================================
%% 2. KONTROLLKLASSEN (Use Cases)
%% ==========================================

class Kontoverwaltung {
    +registrierungAbschliessen(token, name, passwort)
    +anmelden(email, passwort)
    +passwortAendern(athlet, neuesPasswort)
    +passwortResetStart(email)
    +passwortResetBestaetigen(token, neuesPasswort)
}

class AthletenVerwaltung {
    +athletAnlegen(name, email)
    +einladungVersenden(athlet)
    +athletenListe(trainer) : List<Athlet>
}

class TrainingsVerwaltung {
    +trainingErfassen(athlet, daten : Trainingseinheit)
    +trainingAendern(einheit, neueDaten)
    +trainingLoeschen(einheit)
}

class WettbewerbsVerwaltung {
    +wettbewerbDetails(athlet, wettbewerb)
    +wettbewerbErfassen(athlet, daten: Wettbewerb)
    +wettbewerbAendern(einheit, neueDaten)
    +wettbewerbLoeschen(einheit)
}

class AnalyseService {
    +berechneStatistik(athlet, filter : StatistikFilter)
```

```
    +berechneRanking(trainer, filter : RankingFilter)
    +exportiereDaten(athlet, filter) : CSVDatei
}


%% ============================================
%% 3. BEZIEHUNGEN (Assoziationen)
%% ============================================

Trainer "1" --> "0..*" Athlet : verwaltet >
Athlet "1" --> "0..*" Trainingseinheit : hat >
Athlet "1" --> "0..*" Wettbewerb : nimmt teil >

%% ============================================
%% 4. DEPENDENCIES (Kontrollklassen nutzen Entities)
%% ============================================

Kontoverwaltung ..> Athlet : nutzt >
AthletenVerwaltung ..> Athlet : erstellt/liest >
TrainingsVerwaltung ..> Trainingseinheit : verwaltet >
TrainingsVerwaltung ..> Athlet : gehört zu >
WettbewerbsVerwaltung ..> Wettbewerb : liest >
AnalyseService ..> Trainingseinheit : analysiert >
AnalyseService ..> Wettbewerb : analysiert >
AnalyseService ..> StatistikFilter : verwendet >
AnalyseService ..> RankingFilter : verwendet >
AnalyseService ..> CSVDatei : erzeugt >
```

▼ **Legende zum Analyseklassenmodell**

Assoziation (———)

Durchgezogene Linie

- Statische Beziehung zwischen Entitäten
- Beschreibt, wie Objekte dauerhaft zusammengehören
- Mit Multiplizitäten wie `1`, `0..*`
- Beispiel: Athlet — Trainingseinheit

Dependency (..>)

Gestrichelter Pfeil

- Kurzfristige Nutzungsbeziehung
- Kontrollklassen greifen auf Entities zu, ohne sie dauerhaft zu besitzen
- Wird genutzt, um Use Cases abzubilden
- Beispiel: AnalyseService ..> Trainingseinheit

Kontrollklasse (Control)

- Führt Use Cases aus
- Koordiniert Entities
- Enthält Methoden der Geschäftslogik

Entity-Klasse (Entity)

- Repräsentiert fachliche Daten
- Enthält keine Use-Case-Logik
- Hat nur Attribute und evtl. simple Berechnungsfunktionen

▼ UC-01: Kontoverwaltung & UC-07: Passwort zurücksetzen

```
sequenceDiagram
    actor Athlet
    participant UI as :Benutzeroberfläche
    participant KV as :Kontoverwaltung
    participant EM as extern :E-Mail System

    %% --- alt-Block für Login (UC-01) ODER Passwort Res
et (UC-07) ---
    alt Login (Typischer Ablauf UC-01)
        Athlet->>UI: anmelden(email, passwort)
        activate UI
        UI->>KV: anmelden(email, passwort)
        activate KV

        %% Annahme: anmelden() prüft das Passwort und gi
bt OK/Fehler zurück
        KV-->>UI: Authentifizierung OK
```

```
        deactivate KV
        UI-->>Athlet: Anmeldung erfolgreich (Weiterleitu
ng)
        deactivate UI

    else Passwort vergessen (Ablauf UC-07)
        Athlet->>UI: passwortVergessen(email)
        activate UI

        %% Erfüllt A7.1: Funktion starten
        UI->>KV: passwortResetStart(email)
        activate KV

        %% Erfüllt A7.2: E-Mail senden
        KV->>EM: send(email, "Reset-Link")
        EM-->>KV: Mail OK

        KV-->>UI: E-Mail wurde versendet
        deactivate KV
        UI-->>Athlet: Bitte Postfach prüfen
        deactivate UI
    end
```

▼ UC-01: Kontoverwaltung - Profil bearbeiten

```
sequenceDiagram
    autonumber

    actor Athlet
    participant UI as :Benutzeroberfläche
    participant KV as :Kontoverwaltung
    participant A as :Athlet

    %% --- Der Athlet (angemeldet) stößt die Änderung an
---
    Athlet->>UI: profildatenAendern(neuerName, neuesPass
wort)
    activate UI
```

```
    %% --- UI ruft die Kontrollklasse auf ---
    %% Die UI übergibt die Daten an die Business-Logik-K
lasse
    UI->>KV: profildatenAendern(A, neueDaten)
    activate KV

    %% --- Kontrollklasse ruft die Entity-Klasse auf ---
    %% Die Kontoverwaltung ruft die update-Methode des A
thleten-Objekts auf
    KV->>A: updateDaten(neueDaten)
    activate A

    A-->>KV: Daten aktualisiert (OK)
    deactivate A

    %% --- Erfolgsmeldungen zurück an den Athleten ---
    KV-->>UI: Änderung OK
    deactivate KV

    UI-->>Athlet: Profil erfolgreich aktualisiert
    deactivate UI
```

▼ UC-02: Athleten-Einladung versenden

```
sequenceDiagram
    %% --- 1. Aktoren und Objekte definieren ---
    actor Admin
    participant AV as :AthletenVerwaltung
    participant KV as :Kontoverwaltung
    participant EM as extern :E-Mail System

    %% --- 2. Admin stößt den Prozess an ---
    Admin->>AV: athletAnlegen(athletName, athletEmail)
    activate AV

    %% --- 3. Objekt-Erstellung ---
    %% create() erstellt das neue Objekt 'A' vom Typ 'At
```

```
hlet'
    %% Hier wird die Lebenslinie A vom Typ :Athlet dekla
riert
    AV->>A: create(:Athlet)
    participant A as :Athlet
    activate A

    %% --- 4. Alle weiteren Aufrufe gehen an 'A' ---
    AV->>A: setStatus("eingeladen")

    %% --- 5. Interne Logik (Der "Self-Call") ---
    %% Dieser Pfeil ruft eine Methode auf AV auf und übe
rgibt das Objekt A
    AV->>AV: einladungVersenden(A)

    %% --- 6. Aufruf der anderen Kontrollklasse für Mail
---
    %% Dieser Pfeil ruft KV auf und übergibt das 'emai
l'-Attribut von A
    AV->>KV: sendeEinladungsMail(A.email, "gültiger Einl
adungslink")
    activate KV

    %% --- 7. Aufruf des externen Systems ---
    KV->>EM: send(Empfaenger, Betreff, Link)
    EM-->>KV: Mail gesendet (OK)
    deactivate KV

    %% --- 8. Rückmeldungen (gestrichelt) ---
    KV-->>AV: Versand OK
    deactivate A
    deactivate AV

    AV-->>Admin: Athlet erfolgreich angelegt und eingela
den
```

▼ UC-03: Wettbewerbsdaten verwalten

```
sequenceDiagram
    autonumber

    %% Teilnehmer für UC-03
    actor A as Athlet
    participant UI as :Web-UI
    participant WV as :WettbewerbsVerwaltung
    participant W as :Wettbewerb

    %% ==================================================
=====
    %% WETTBEWERB ANLEGEN
    %% ==================================================
=====
    A ->> UI: neuenWettbewerbEingeben(Datum, Name, Sport
art, Dauer)
    UI ->> WV: wettbewerbErfassen(athlet, daten)
    activate WV

    WV ->> W: create(daten)
    activate W
    W -->> WV: Wettbewerb erstellt
    deactivate W

    WV -->> UI: Speichern OK
    deactivate WV

    UI -->> A: Wettbewerb erfolgreich angelegt

    %% ==================================================
=====
    %% WETTBEWERB BEARBEITEN
    %% ==================================================
=====
    A ->> UI: wettbewerbBearbeiten(einheit)
    UI ->> WV: wettbewerbAendern(einheit, neueDaten)
    activate WV
```

```
    WV ->> W: update(neueDaten)
    activate W
    W -->> WV: Änderung gespeichert
    deactivate W

    WV -->> UI: Änderung OK
    deactivate WV

    UI -->> A: Wettbewerb aktualisiert

    %% ==================================================
=====
    %% WETTBEWERB LÖSCHEN
    %% ==================================================
=====
    A ->> UI: wettbewerbLoeschen(einheit)
    UI ->> WV: wettbewerbLoeschen(einheit)
    activate WV

    WV ->> W: delete()
    activate W
    W -->> WV: gelöscht
    deactivate W

    WV -->> UI: Löschung OK
    deactivate WV

    UI -->> A: Wettbewerb gelöscht
```

▼ UC-04: Trainingseinheit verwalten

```
sequenceDiagram
    autonumber

    %% Teilnehmer wie im Beispiel
    actor A as Athlet
    participant UI as :Web-UI
    participant TV as :TrainingsVerwaltung
```

```
    participant TE as :Trainingseinheit

    %% ==================================================
=====
    %% TRAINING ANLEGEN
    %% ==================================================
=====
    A ->> UI: neueTrainingseinheitEingeben(Datum, Sporta
rt, Dauer, Distanz)
    UI ->> TV: trainingErfassen(athlet, daten)
    activate TV

    TV ->> TE: create(daten)
    activate TE
    TE -->> TV: Trainingseinheit erstellt
    deactivate TE

    TV -->> UI: Speichern OK
    deactivate TV

    UI -->> A: Trainingseinheit erfolgreich angelegt

    %% ==================================================
=====
    %% TRAINING BEARBEITEN
    %% ==================================================
=====
    A ->> UI: trainingseinheitBearbeiten(einheit)
    UI ->> TV: trainingAendern(einheit, neueDaten)
    activate TV

    TV ->> TE: update(neueDaten)
    activate TE
    TE -->> TV: Änderung gespeichert
    deactivate TE

    TV -->> UI: Änderung OK
    deactivate TV
```

```
    UI -->> A: Trainingseinheit aktualisiert

    %% ==================================================
=====
    %% TRAINING LÖSCHEN
    %% ==================================================
=====
    A ->> UI: trainingseinheitLoeschen(einheit)
    UI ->> TV: trainingLoeschen(einheit)
    activate TV

    TV ->> TE: delete()
    activate TE
    TE -->> TV: gelöscht
    deactivate TE

    TV -->> UI: Löschung OK
    deactivate TV

    UI -->> A: Trainingseinheit gelöscht
```

▼ UC-05: Persönliche Statistiken & UC-08: Trainingsdaten exportieren

```
sequenceDiagram
    actor Athlet
    participant UI as :Benutzeroberfläche
    participant AS as :AnalyseService
    participant SF as :StatistikFilter
    participant DB as :Datenbank
    participant CSV as :CSVDatei

    %% --- 1. Statistik laden (UC-05) ---
    Athlet->>UI: statistikAnzeigen(Kriterien)
    activate UI

    %% Erfüllt A5.2: Filtern und anzeigen
    UI->>SF: create(Kriterien)
```

```
    UI->>AS: berechneStatistik(Athlet, filter)
    activate AS

    AS->>DB: findeLeistungsdaten(Athlet, filter)
    activate DB
    DB-->>AS: Liste<Training/Wettbewerb>
    deactivate DB

    AS->>AS: erstelleStatistik(daten)

    AS-->>UI: Statistik-Daten
    deactivate AS
    UI-->>Athlet: Statistik wird angezeigt
    deactivate UI

    %% --- 2. Daten exportieren (UC-08) ---
    Athlet->>UI: exportiereMeineDaten()
    activate UI

    %% Erfüllt A8.1: Daten exportieren
    UI->>AS: exportiereDaten(Athlet, filter)
    activate AS

    AS->>DB: findeLeistungsdaten(Athlet, filter)
    activate DB
    DB-->>AS: Liste<Training/Wettbewerb>
    deactivate DB

    %% Erzeugt die CSV-Klasse aus eurem Klassendiagramm
    AS->>CSV: create(Listen-Daten)

    AS-->>UI: :CSVDatei
    deactivate AS
    UI-->>Athlet: Download der CSV-Datei startet
    deactivate UI
```

▼ UC-06: Rankinganalyse und Vergleich durchführen

```
sequenceDiagram
    actor Trainer
    participant UI as :Benutzeroberfläche
    participant AS as :AnalyseService
    participant RF as :RankingFilter
    participant DB as :Datenbank

    alt Rankingdaten berechnen
        Trainer->>UI: Ranking-Analyse starten (Kriterien
wählen)
        activate UI



        UI->>RF: create(Kriterien)

        UI->>AS: berechneRanking(Trainer, filter: Rankin
gFilter)
        activate AS



        AS->>DB: findeAthletenDesTrainers(Trainer)
        activate DB
        DB-->>AS: Liste<Athlet>
        deactivate DB



        AS->>DB: findeLeistungsdaten(Liste<Athlet>, filt
er: RankingFilter)
        activate DB
        DB-->>AS: Liste<Trainingseinheit/Wettbewerb>
        deactivate DB



        AS->>AS: erstelleRankingListe(Leistungsdaten)
```

```
        AS-->>UI: Ranking-Daten (Liste<RankingEintrag>)
        deactivate AS

        UI-->>Trainer: Ranking-Liste anzeigen
        deactivate UI
    else Alternative A1: Keine Daten gefunden
        Trainer->>UI: Ranking-Analyse starten (Kriterien
wählen)
        activate UI
        UI->>RF: create(Kriterien)

        UI->>AS: berechneRanking(Trainer, filter: Rankin
gFilter)
        activate AS

        AS->>DB: findeLeistungsdaten(...)
        activate DB
        DB-->>AS: Leere Liste<Leistungsdaten>
        deactivate DB

        AS-->>UI: Fehlermeldung/leere Daten
        deactivate AS
        UI-->>Trainer: Meldung anzeigen: "Keine Daten fü
r die gewählten Kriterien gefunden."
        deactivate UI
    end
```

Anforderungsverfolgung

Anforderungs-ID (Quelle)	Kurzbeschreibung (aus anforderungen.pdf)	Umsetzung im Klassendiagramm (Ziel)
A1.1	Registrierung abschließen	`Klasse: Kontoverwaltung` `Methode: +registrierungAbschliessen(...)`
A1.2	Anmeldung (Authentifizierung)	`Klasse: Kontoverwaltung` `Methode: +anmelden(...)`

A1.3	Profil (Passwort) ändern	`Klasse: Kontoverwaltung` `Methode: +passwortAendern(...)`

A2.1	Admin: Athleten verwalten, anlegen & einladen	`Klasse: AthletenVerwaltung`
		`Methode: +athletAnlegen(...)`
		`Methode: +einladungVersenden(...)`

A2.2	Admin: Athleten-Übersicht anzeigen	`Klasse: AthletenVerwaltung` `Methode: +athletenListe(...)`
A3.1	Wettbewerbsdaten "einsehen und Verwalten"	`Klasse: WettbewerbsVerwaltung` `Methode: +wettbewerbDetails(...)`
A4.1	Trainingseinheit erfassen	`Klasse: TrainingsVerwaltung` `Methode: +trainingErfassen(...)`
A4.2	Trainingseinheit ändern	`Klasse: TrainingsVerwaltung` `Methode: +trainingAendern(...)`
A4.3	Trainingseinheit löschen	`Klasse: TrainingsVerwaltung` `Methode: +trainingLoeschen(...)`
A5.1	Persönliche Statistik-Seite öffnen	`Klasse: AnalyseService` `Methode: +berechneStatistik(...)`

A5.2	Persönliche Statistik filtern/anzeigen	`Klasse: AnalyseService`
		`Methode: +berechneStatistik(...)`
		`Klasse: StatistikFilter`

A6.1	Rankinganalyse starten/filtern	`Klasse: AnalyseService`
		`Methode: +berechneRanking(...)`
		`Klasse: RankingFilter`

A6.2	Ranking: Fehlermeldung (keine Daten)	`Klasse: AnalyseService` `Methode: +berechneRanking(...)`
A6.3	Ranking: Automatische Aktualisierung	`Klasse: AnalyseService` `Methode: +berechneRanking(...)`
A7.1	Passwort-Reset-Prozess starten	`Klasse: Kontoverwaltung` `Methode: +passwortResetStart(...)`
A7.2	Passwort-Reset-Link senden	`Klasse: Kontoverwaltung` `Methode: +passwortResetStart(...)`
A7.3	Neues Passwort vergeben	`Klasse: Kontoverwaltung` `Methode: +passwortResetBestaetigen(...)`

A8.1	Trainingsdaten exportieren (Athlet)	`Klasse: AnalyseService`
		`Methode: +exportiereDaten(...)`
		`Klasse: CSVDatei`